

Furkan_Sağlam_2526630

December 14, 2025

1 Assignment 3: Hidden Markov Models for POS Tagging and Text Generation

1.1 CNG463 - Introduction to Natural Language Processing

1.1.1 METU NCC Computer Engineering | Fall 2025-26

Student Name: Furkan Sağlam

Student ID: 2526630 **Due Date:** 14 December 2025 (Sunday) before midnight

1.2 Overview

This assignment focuses on: 1. Building **supervised**, **unsupervised**, and **semi-supervised** HMM models for Part-of-Speech (POS) tagging 2. Implementing **5-fold cross-validation** to evaluate model performance 3. Comparing the three training approaches using per-tag and overall accuracy 4. Using HMMs for **creative text generation** with temperature-based sampling

Note: You will use the Brown corpus with universal POS tagset. Start with 5000 sentences for debugging, then run on the full corpus (or as much as Colab can handle).

Grading: - Helper Functions: **10 pts** - `remove_tags()`: 2 pts - `evaluate_tagger()`: 5 pts - Results display: 3 pts - Semi-supervised HMM: **20 pts** - 5-fold cross-validation: **25 pts** - Supervised HMM: 10 pts - Unsupervised HMM (Baum-Welch): 10 pts - Semi-supervised HMM - Evaluate and Accumulate Results: 5 pts - Report on Results: **5 pts** - Text Generation: **24 pts** - `sample_state()`: 7 pts - `sample_word()`: 7 pts - `generate_text_from_word()`: 10 pts - Written Questions (4×4 pts): **16 pts** - **Total: 100 pts**

1.3 Pre-Submission Checklist

- ☐ Name and student ID at top
- ☐ No cells are added or removed
- ☐ All TODO sections completed
- ☐ All questions answered
- ☐ Code runs without errors
- ☐ Results tables included
- ☐ Run All before saving

1.4 Setup and Imports

```
[3]: # Standard libraries
import numpy as np
import random
from collections import defaultdict

# NLTK for corpus and HMM
import nltk
from nltk.tag import hmm

# Scikit-learn for cross-validation
from sklearn.model_selection import KFold

# Set random seed for reproducibility
seed = 42
np.random.seed(seed)
random.seed(seed)
```

2 Task 1: HMM-based POS Tagging (72 points)

In this part, you will implement three different approaches to training HMM models for POS tagging: 1. **Supervised learning**: Uses fully labelled data 2. **Unsupervised learning**: Uses unlabelled data with Baum-Welch algorithm 3. **Semi-supervised learning**: Combines a small amount of labelled data with larger unlabelled data

2.1 1.1: Load the Brown Corpus

Load the Brown corpus with universal POS tagset. Start with 5000 sentences for testing, then increase to the maximum your environment can handle.

```
[1]: # Download required NLTK data
import nltk
from nltk.corpus import brown
nltk.download('brown')
nltk.download('universal_tagset')

NUM_SENTENCES = 5000 # Start with 5000 sentences and increase later

all_data = list(brown.tagged_sents(tagset="universal"))[:5000]

print(f"Total sentences loaded: {len(all_data)}")
print(f"\nExample sentence:")
print(all_data[0])
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data] Unzipping taggers/universal_tagset.zip.
```

Total sentences loaded: 5000

Example sentence:

```
[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'),
('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'),
('investigation', 'NOUN'), ('of', 'ADP'), ('Atlanta's', 'NOUN'), ('recent',
'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('`',
'.'), ('no', 'DET'), ('evidence', 'NOUN'), ('"', '.'), ('that', 'ADP'), ('any',
'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.',
'.')] ]
```

2.2 1.2: Remove Tags (2 points)

Implement the `remove_tags()` function that converts tagged sentences to untagged format required by NLTK's unsupervised training.

Input: List of tagged sentences `[(word, tag), ...], ...]`

Output: List of untagged sentences `[(word, None), ...], ...]`

```
[2]: def remove_tags(tagged_sents):
    """
    Remove tags from tagged sentences to create untagged data.

    Args:
        tagged_sents: List of tagged sentences [(word, tag), ...], ...]

    Returns:
        List of untagged sentences [(word, None), ...], ...] (required format_
    ↪ for NLTK)
    """
    # TODO: Implement this function
    untagged = []
    for sent in tagged_sents:
        untagged_sent = []
        for word, tag in sent:
            untagged_sent.append((word, None))
        untagged.append(untagged_sent)

    return untagged
# Corrected test data to match the expected input format of the function
test=[(['furkan', 'name'], ('the', 'pro')), ([('Life', 'name'))]]; # Wrapped the list_
↪ of tuples in another list to represent a single sentence
test2=remove_tags(test)
print(test2)
```

```
[(['furkan', None), ('the', None)], [(['Life', None)])]
```

2.3 1.3: Evaluate Tagger (5 points)

Implement the `evaluate_tagger()` function that evaluates a trained tagger on test data and returns per-tag accuracy statistics.

Input: - `tagger`: Trained HMM tagger - `test_data`: List of tagged test sentences

Output: Dictionary with per-tag statistics {tag: {'correct': count, 'total': count}}

```
[4]: def evaluate_tagger(tagger, test_data):
    """
    Evaluate tagger and return per-tag accuracy.

    Args:
        tagger: Trained HMM tagger
        test_data: List of tagged sentences for testing

    Returns:
        dict of {tag: {'correct': count, 'total': count}}
    """
    # TODO: Implement this function
    #
    # Steps:
    # 1. Initialize tag_stats as defaultdict(lambda: {'correct': 0, 'total': 0})
    # 2. For each sentence in test_data:
    #     a. Extract words and true tags
    #     b. Use tagger.tag(words) to get predicted tags
    #     c. Compare true vs predicted tags
    #     d. Update tag_stats accordingly
    # 3. Handle exceptions (if tagging fails, count all as incorrect)
    # 4. Return tag_stats

    tag_stats = defaultdict(lambda: {'correct': 0, 'total': 0})

    words = []
    true_tags = []
    for sent in test_data:
        words = [w for w, t in sent]
        true_tags = [t for w, t in sent]
        try:
            predicted_tags = tagger.tag(words)
        except Exception:
            for true_tag in true_tags:
                tag_stats[true_tag]['total'] += 1
            continue

        for i in range(len(predicted_tags)):
```

```

    true_tag = true_tags[i]
    if predicted_tags[i][1] == true_tags[i]:
        tag_stats[true_tag]['correct'] += 1

    tag_stats[true_tag]['total'] += 1

return tag_stats

```

2.4 1.4: Semi-supervised HMM Training (20 points)

Implement semi-supervised HMM training that combines a small amount of labelled data (1%) with larger unlabelled data (99%).

Steps: 1. Split data: 1% tagged, 99% untagged 2. Train supervised model on 1% tagged data 3. Use this model to initialise Baum-Welch on 99% untagged data 4. Refine with unsupervised learning

```

[8]: def train_semi_supervised(tagged_data, percent_tagged=1.0):
    """
    Train HMM with a mix of tagged and untagged data.

    Args:
        tagged_data: List of tagged sentences
        percent_tagged: Percentage of data to keep tags (default 1.0%)

    Returns:
        Trained HMM tagger
    """
    # TODO: Implement semi-supervised training
    #
    # Steps:
    # 1. Calculate split index: int(len(tagged_data) * (percent_tagged / 100.0))
    #     Ensure at least 1 sentence is tagged
    #
    # 2. Split data:
    #     tagged_portion = tagged_data[:split_idx]
    #     untagged_portion = remove_tags(tagged_data[split_idx:])
    #
    # 3. Extract states and symbols from ALL data
    #     NOTE: In fact, we should have used only the tagged_data for this,
    #     but we are not sure if that portion includes all the tags and symbols
    #
    # 4. Initialize trainer with states and symbols using
    #     hmm.HiddenMarkovModelTrainer
    #
    # 5. Train supervised model on small tagged data using trainer.
    ↪ train_supervised

```

```

#
# 6. Refine with Baum-Welch on untagged data using trainer.
↪train_unsupervised
#
# 7. Return refined_tagger (or error if no untagged data or no tagged data)

splitindex = int(len(tagged_data) * (percent_tagged / 100.0))
if splitindex < 1:
    splitindex = 1

tagged_portion = tagged_data[:splitindex]
untagged_portion = remove_tags(tagged_data[splitindex:])

states = []
symbols = []
for sent in tagged_data:
    for word, tag in sent:
        states.append(tag)
        symbols.append(word)

states = list(states)
symbols = list(symbols)
states = set(states)
symbols = set(symbols)

trainer = hmm.HiddenMarkovModelTrainer(states,symbols)

if not tagged_portion:
    raise ValueError("No tagged data available for supervised training.")

supervised_tagger = trainer.train_supervised(tagged_portion)

if untagged_portion:
    refined_tagger = trainer.
↪train_unsupervised(untagged_portion,model=supervised_tagger,
↪max_iterations=10)
    return refined_tagger
else:
    return supervised_tagger

```

2.5 1.5: 5-Fold Cross-Validation (25 Points)

Implement 5-fold cross-validation to train and evaluate all three models. This ensures robust performance estimates.

Steps: 1. Split data into 5 folds 2. For each fold: - Train - supervised (trainer.train_supervised()) - unsupervised (trainer.train_unsupervised()) - semi-supervised models (train_semi_supervised()) - Evaluate on test fold - Accumulate results 3.

Calculate average accuracy across all folds

```
[10]: kf = KFold(n_splits=5, shuffle=True, random_state=seed)

# Storage for results across folds
results = {
    'unsupervised': defaultdict(lambda: {'correct': 0, 'total': 0}),
    'supervised': defaultdict(lambda: {'correct': 0, 'total': 0}),
    'semi_supervised': defaultdict(lambda: {'correct': 0, 'total': 0})
}

# TODO: Complete the cross-validation loop
#
# for train_idx, test_idx in kf.split(all_data):
#     print(f"{'='*60}")
#     print(f"FOLD {fold_num}")
#     print(f"{'='*60}")
#
#     # Split data to train_data and test_data
#     ...
#
#     print(f"Training set: {len(train_data)} sentences")
#     print(f"Test set: {len(test_data)} sentences")
#
#     # Extract all_tags and all_symbols from training data
#     ...
#
#
#     # 1. Unsupervised HMM (Baum-Welch)
#     print("\n1. Training Unsupervised HMM (Baum-Welch)...")
#     try:
#         1.1 Convert to untagged format
#
#         1.2 Initialize trainer with states and symbols
#
#         1.3 Train unsupervised
#
#         1.4 Evaluate unsupervised tagger and update the results dict
#
#         print("Unsupervised training complete")
#     except Exception as e:
#         print(f"ERROR: Unsupervised training failed: {e}")
#         import traceback
#         traceback.print_exc()
#
#
#     # 2. Supervised HMM
#     print("\n2. Training Supervised HMM...")
```

```

#     try:
#         1.1 Initialize trainer with states and symbols
#
#         1.2 Train supervised
#
#         1.3 Evaluate supervised tagger and update the results dict
#
#         print("Supervised training complete")
#     except Exception as e:
#         print(f"ERROR: Supervised training failed: {e}")
#         import traceback
#         traceback.print_exc()
#
#     # 3. Semi-supervised HMM (1% tagged, 99% untagged)
#     print("\n3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...")
#     try:
#         1.1 Train semi-supervised
#
#         1.3 Evaluate semi-supervised tagger and update the results dict
#
#         print("Semi-supervised training complete")
#     except Exception as e:
#         print(f"ERROR: Semi-supervised training failed: {e}")
#         import traceback
#         traceback.print_exc()
fold_num=1

for train_idx, test_idx in kf.split(all_data):
    print(f"{'='*60}")
    print(f"FOLD {fold_num}")
    print(f"{'='*60}")
    train_data = [all_data[i] for i in train_idx]
    test_data = [all_data[i] for i in test_idx]
    print(f"Training set: {len(train_data)} sentences")
    print(f"Test set: {len(test_data)} sentences")
    # Extract all_tags and all_symbols from training data
    states = []
    symbols = []
    for sent in train_data:
        for word, tag in sent:
            states.append(tag)
            symbols.append(word)
    all_tags = list(set(states))
    all_symbols = list(set(symbols))
    #     # 1. Unsupervised HMM (Baum-Welch)
    print("\n1. Training Unsupervised HMM (Baum-Welch)...")
    try:

```

```

untagged = remove_tags(train_data)
trainer = hmm.HiddenMarkovModelTrainer(all_tags,all_symbols)
unsupervised_tagger = trainer.
↳train_unsupervised(untagged,max_iterations=15)
stast = evaluate_tagger(unsupervised_tagger, test_data)
for tag, stats in stast.items():
    results['unsupervised'][tag]['correct'] += stats['correct']
    results['unsupervised'][tag]['total'] += stats['total']

print("Unsupervised training complete")
except Exception as e:
    print(f"ERROR: Unsupervised training failed: {e}")
    import traceback
    traceback.print_exc()
    # 2. Supervised HMM

print("\n2. Training Supervised HMM...")
try:
    trainer = hmm.HiddenMarkovModelTrainer(all_tags,all_symbols)
    supervised_tagger = trainer.train_supervised(train_data)
    stast = evaluate_tagger(supervised_tagger,test_data)
    for tag, stats in stast.items():
        results['supervised'][tag]['correct'] += stats['correct']
        results['supervised'][tag]['total'] += stats['total']
    print("Supervised training complete")
except Exception as e:
    print(f"ERROR: Supervised training failed: {e}")
    import traceback
    traceback.print_exc()
print("\n3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...")
try:
    semisupervised_tagger =
↳train_semi_supervised(train_data,percent_tagged=10)
    stast = evaluate_tagger(semisupervised_tagger,test_data)
    for tag, stats in stast.items():
        results['semi_supervised'][tag]['correct'] += stats['correct']
        results['semi_supervised'][tag]['total'] += stats['total']

    print("Semi-supervised training complete")
except Exception as e:
    print(f"ERROR: Semi-supervised training failed: {e}")
    import traceback
    traceback.print_exc()
fold_num = fold_num+1

```

=====

FOLD 1

=====

Training set: 4000 sentences

Test set: 1000 sentences

1. Training Unsupervised HMM (Baum-Welch)...

```
iteration 0 logprob -1190642.074199525
iteration 1 logprob -878286.4256169131
iteration 2 logprob -876988.9021594055
iteration 3 logprob -875742.1353678013
iteration 4 logprob -874274.9947157332
iteration 5 logprob -872369.4217657886
iteration 6 logprob -869791.7831451256
iteration 7 logprob -866289.6529867788
iteration 8 logprob -861657.7902753542
iteration 9 logprob -855882.2163915882
iteration 10 logprob -849181.1488600201
iteration 11 logprob -841858.9171849577
iteration 12 logprob -834430.0716174529
iteration 13 logprob -827541.1208543386
iteration 14 logprob -821694.799850572
Unsupervised training complete
```

2. Training Supervised HMM...

```
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335: RuntimeWarning:
overflow encountered in cast
```

```
    O[i, k] = self._output_logprob(si, self._symbols[k])
```

```
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:333: RuntimeWarning:
overflow encountered in cast
```

```
    X[i, j] = self._transitions[si].logprob(self._states[j])
```

```
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:363: RuntimeWarning:
overflow encountered in cast
```

```
    O[i, k] = self._output_logprob(si, self._symbols[k])
```

Supervised training complete

3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...

```
iteration 0 logprob -2.3523000000000316e+304
iteration 1 logprob -1.106911594400326e+290
iteration 2 logprob -861770.0633351952
iteration 3 logprob -763534.9722836794
iteration 4 logprob -751470.6757283926
iteration 5 logprob -741101.5868531963
iteration 6 logprob -733668.0853519735
iteration 7 logprob -729053.9844446966
iteration 8 logprob -726411.6746079717
iteration 9 logprob -724899.683635057
iteration 10 logprob -723978.4867967915
```

```

iteration 11 logprob -723394.5497278719
iteration 12 logprob -723013.3051277914
iteration 13 logprob -722750.6887054092
iteration 14 logprob -722558.8623387157

/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:331: RuntimeWarning:
overflow encountered in cast
  P[i] = self._priors.logprob(si)

Semi-supervised training complete
=====
FOLD 2
=====
Training set: 4000 sentences
Test set: 1000 sentences

1. Training Unsupervised HMM (Baum-Welch)...
iteration 0 logprob -1186503.8648538345
iteration 1 logprob -876277.0738886857
iteration 2 logprob -874890.2751688311
iteration 3 logprob -873565.6937796626
iteration 4 logprob -872050.8987730428
iteration 5 logprob -870152.3389175783
iteration 6 logprob -867689.6284707934
iteration 7 logprob -864513.9451048966
iteration 8 logprob -860599.1575679354
iteration 9 logprob -856071.4347542449
iteration 10 logprob -850908.5157831605
iteration 11 logprob -844899.5361411602
iteration 12 logprob -838223.9737109793
iteration 13 logprob -831525.0563244976
iteration 14 logprob -825466.9761231008
Unsupervised training complete

2. Training Supervised HMM...
Supervised training complete

3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...
iteration 0 logprob -2.3820000000000345e+304
iteration 1 logprob -1.0939210146878642e+290
iteration 2 logprob -858918.3287111203
iteration 3 logprob -766351.2667053686
iteration 4 logprob -754987.6626063645
iteration 5 logprob -744005.873490008
iteration 6 logprob -735479.2766314765
iteration 7 logprob -729997.151182666
iteration 8 logprob -726882.6710524695
iteration 9 logprob -725174.0712541026
iteration 10 logprob -724174.5346130752

```

```

iteration 11 logprob -723540.8359423921
iteration 12 logprob -723114.2504544806
iteration 13 logprob -722811.2875887763
iteration 14 logprob -722585.1293140264
Semi-supervised training complete
=====
FOLD 3
=====
Training set: 4000 sentences
Test set: 1000 sentences

1. Training Unsupervised HMM (Baum-Welch)...
iteration 0 logprob -1199310.7511336354
iteration 1 logprob -883962.696157659
iteration 2 logprob -881944.780437931
iteration 3 logprob -880120.7664980459
iteration 4 logprob -878087.690012097
iteration 5 logprob -875612.2572870932
iteration 6 logprob -872569.6943498407
iteration 7 logprob -868969.335967225
iteration 8 logprob -864870.3553656297
iteration 9 logprob -860166.5462927864
iteration 10 logprob -854663.5526917533
iteration 11 logprob -848412.5871947366
iteration 12 logprob -841853.9333376577
iteration 13 logprob -835618.3146072923
iteration 14 logprob -830195.0536196496
Unsupervised training complete

2. Training Supervised HMM...
Supervised training complete

3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...
iteration 0 logprob -2.352600000000036e+304
iteration 1 logprob -1.1017843600998947e+290
iteration 2 logprob -866455.6669921958
iteration 3 logprob -770414.8690294721
iteration 4 logprob -758843.2482859669
iteration 5 logprob -748736.2072598123
iteration 6 logprob -741247.9708558307
iteration 7 logprob -736467.2227133291
iteration 8 logprob -733662.6430959295
iteration 9 logprob -732027.7472839688
iteration 10 logprob -731018.8071645442
iteration 11 logprob -730353.4302674924
iteration 12 logprob -729900.0481092064
iteration 13 logprob -729580.964111565
iteration 14 logprob -729345.9725314028

```

Semi-supervised training complete

=====

FOLD 4

=====

Training set: 4000 sentences

Test set: 1000 sentences

1. Training Unsupervised HMM (Baum-Welch)...

iteration 0 logprob -1190481.246810738
iteration 1 logprob -878068.1728553072
iteration 2 logprob -877059.6691692701
iteration 3 logprob -875993.1186185036
iteration 4 logprob -874724.7239413385
iteration 5 logprob -873091.0243345717
iteration 6 logprob -870877.4672133464
iteration 7 logprob -867804.5121303797
iteration 8 logprob -863587.3044481234
iteration 9 logprob -858172.3776489608
iteration 10 logprob -851981.7262384645
iteration 11 logprob -845533.6469916786
iteration 12 logprob -839022.3566576241
iteration 13 logprob -832769.6216414542
iteration 14 logprob -827187.9958148018

Unsupervised training complete

2. Training Supervised HMM...

Supervised training complete

3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...

iteration 0 logprob -2.302200000000034e+304
iteration 1 logprob -1.1282889494766027e+290
iteration 2 logprob -857222.3532968945
iteration 3 logprob -762003.9842777165
iteration 4 logprob -750819.1577458649
iteration 5 logprob -741048.1260800086
iteration 6 logprob -733696.4508686563
iteration 7 logprob -728970.0691858798
iteration 8 logprob -726148.9224437206
iteration 9 logprob -724472.6796382939
iteration 10 logprob -723437.1754342542
iteration 11 logprob -722748.2687442383
iteration 12 logprob -722274.8681480188
iteration 13 logprob -721945.1602268282
iteration 14 logprob -721713.1044065977

Semi-supervised training complete

=====

FOLD 5

=====

Training set: 4000 sentences
Test set: 1000 sentences

1. Training Unsupervised HMM (Baum-Welch)...

```
iteration 0 logprob -1199226.8273591143
iteration 1 logprob -884390.7573862269
iteration 2 logprob -882614.9330998282
iteration 3 logprob -880953.5568615722
iteration 4 logprob -879123.0205123911
iteration 5 logprob -876915.5945280943
iteration 6 logprob -874215.6387094925
iteration 7 logprob -871095.6362963456
iteration 8 logprob -867765.3790952706
iteration 9 logprob -864136.8871441175
iteration 10 logprob -859837.5958120956
iteration 11 logprob -854681.0979625295
iteration 12 logprob -848788.8792996069
iteration 13 logprob -842550.3914064843
iteration 14 logprob -836502.3418944745
Unsupervised training complete
```

2. Training Supervised HMM...

Supervised training complete

3. Training Semi-Supervised HMM (1% tagged, 99% untagged)...

```
iteration 0 logprob -2.358200000000035e+304
iteration 1 logprob -1.0936949881177755e+290
iteration 2 logprob -865368.5888233292
iteration 3 logprob -770485.8697315867
iteration 4 logprob -758770.9977549334
iteration 5 logprob -748488.852241613
iteration 6 logprob -740954.8409417067
iteration 7 logprob -736219.51642891
iteration 8 logprob -733443.0592781777
iteration 9 logprob -731799.764888804
iteration 10 logprob -730756.4784365455
iteration 11 logprob -730038.791607177
iteration 12 logprob -729531.0355069522
iteration 13 logprob -729167.7754491802
iteration 14 logprob -728899.5110404986
Semi-supervised training complete
```

[]:

Question 1.1: In an unsupervised HMM with 12 hidden states, you replace the intended PoS tags with meaningless labels like S1–S12. How would this change affect the model’s learned behaviour and its evaluation accuracy? (4 points, 3-4 sentences)

\

[In unsupervised HMM change meaningless labels, doesn't affect learn model. Model still learns patterns from groups, but these states don't have linguistic meaning. For these reason, accuracy of evaluation decreases.]

Question 1.2: You initialize a semi-supervised HMM with a fully supervised model and then run Baum–Welch only on unlabeled data. How can this additional unsupervised training change the model's behaviour and accuracy? (4 points, 3-4 sentences)

[Initializing with a supervised model allows the HMM to start with close parameters and the correct structure. Unsupervised training can adapt to the broader data patterns and improve generalization. However, the model can stray from true tags because of overuse.]

Question 1.3: Consider the following four changes to an unsupervised or semi-supervised HMM for PoS tagging. For each one, state whether it would make training faster, slower, or have no significant effect, and justify your choice in one sentence. (4 points, 4 sentences) - Reducing the number of hidden states from 12 to 6. - Initializing the model with a fully supervised HMM instead of random parameters. - Increasing the size of the unlabeled corpus by a factor of 5. - Replacing full EM with a fixed small number of EM iterations (e.g., exactly 3 passes).

- Reducing the number of hidden states from 12 to 6 makes training faster, because reduce number of parameters and computations
- Initializing the model with a fully supervised HMM instead of random parameters makes faster because the model has good parameters.
- Increasing the size of the unlabeled corpus by a factor of 5 makes training slower because must process more data for each iteration.
- Replacing full EM with a fixed small number of EM iterations makes training faster because the algorithm stops early instead of until convergence.

2.6 1.6: Display Results (5 points)

Display the average across folds results in a formatted table showing - tag frequency (%) - per-tag accuracy (%) - overall accuracy (%)

1. List item
2. List item

for all three models.

```
[13]: # TODO: Calculate and display results
#

print("Model | Tag | Tag Frequency (%) | Per-Tag Accuracy (%) | Overall_
Accuracy (%)")

for modelname in results:
    tags = results[modelname]
```

```

total_correct = 0
total_tokens = 0
for tag in tags:
    total_correct += tags[tag]['correct']
    total_tokens += tags[tag]['total']
total_accuracy = (total_correct / total_tokens)*100
for tag in tags:
    tag_correct = tags[tag]['correct']
    tag_total = tags[tag]['total']
    tag_freq = (tag_total / total_tokens) * 100
    tag_acc = (tag_correct / tag_total) * 100
    print(
        f"{modelname:15} | {tag:5} | "
        f"{tag_freq:16.2f} | "
        f"{tag_acc:20.2f} | "
        f"{total_accuracy:19.2f}"
    )

```

Model	Tag	Tag Frequency (%)	Per-Tag Accuracy (%)	Overall Accuracy (%)
unsupervised 22.61	ADV	3.42	3.44	
unsupervised 22.61	.	11.68	0.25	
unsupervised 22.61	DET	11.42	5.93	
unsupervised 22.61	NOUN	30.17	65.39	
unsupervised 22.61	VERB	14.47	3.01	
unsupervised 22.61	PRON	2.56	5.64	
unsupervised 22.61	NUM	2.06	4.91	
unsupervised 22.61	PRT	2.25	2.29	
unsupervised 22.61	ADJ	6.82	2.66	
unsupervised 22.61	CONJ	2.73	0.37	
unsupervised 22.61	ADP	12.32	9.18	
unsupervised 22.61	X	0.09	2.02	
supervised 59.00	ADV	3.42	42.86	
supervised 59.00	.	11.68	33.45	

supervised	DET		11.42		49.04	
59.00						
supervised	NOUN		30.17		99.41	
59.00						
supervised	VERB		14.47		44.69	
59.00						
supervised	PRON		2.56		56.18	
59.00						
supervised	NUM		2.06		39.58	
59.00						
supervised	PRT		2.25		40.87	
59.00						
supervised	ADJ		6.82		36.08	
59.00						
supervised	CONJ		2.73		35.10	
59.00						
supervised	ADP		12.32		40.20	
59.00						
supervised	X		0.09		25.25	
59.00						
semi_supervised	ADV		3.42		31.20	
50.81						
semi_supervised	.		11.68		30.91	
50.81						
semi_supervised	DET		11.42		45.07	
50.81						
semi_supervised	NOUN		30.17		88.63	
50.81						
semi_supervised	VERB		14.47		34.21	
50.81						
semi_supervised	PRON		2.56		50.68	
50.81						
semi_supervised	NUM		2.06		24.77	
50.81						
semi_supervised	PRT		2.25		28.87	
50.81						
semi_supervised	ADJ		6.82		24.30	
50.81						
semi_supervised	CONJ		2.73		30.24	
50.81						
semi_supervised	ADP		12.32		35.28	
50.81						
semi_supervised	X		0.09		3.03	
50.81						

2.7 1.7: Sample Predictions

Test the models on sample sentences to see how it performs.

```

[16]: tagger_sup = supervised_tagger
tagger_unsup = unsupervised_tagger
tagger_semi_sup = semisupervised_tagger
# Print sample predictions from the last fold
print("Sample predictions from last fold:")
sample_sentences = [
    "Today is a good day .",
    "Joe met Joanne in Delhi .",
    "Time flies like an arrow .",
    "The good , the bad , the ugly went to a bar ."
]

print(f"{'='*10} Spervised HMM Tagger {'='*10}")
for sent in sample_sentences:
    try:
        tagged = tagger_sup.tag(sent.split())
        print(f" {sent}")
        print(f" → {tagged}\n")
    except Exception as e:
        print(f" {sent}")
        print(f"ERROR: Tagging failed: {e}\n")

print(f"{'='*10} Unspervised HMM Tagger {'='*10}")
for sent in sample_sentences:
    try:
        tagged = tagger_unsup.tag(sent.split())
        print(f" {sent}")
        print(f" → {tagged}\n")
    except Exception as e:
        print(f" {sent}")
        print(f"ERROR: Tagging failed: {e}\n")

print(f"{'='*10} Semi-spervised HMM Tagger {'='*10}")
for sent in sample_sentences:
    try:
        tagged = tagger_semi_sup.tag(sent.split())
        print(f" {sent}")
        print(f" → {tagged}\n")
    except Exception as e:
        print(f" {sent}")
        print(f"ERROR: Tagging failed: {e}\n")

```

Sample predictions from last fold:

===== Spervised HMM Tagger =====

```

    Today is a good day .
    → [('Today', 'NOUN'), ('is', 'VERB'), ('a', 'DET'), ('good', 'ADJ'), ('day',
'NOUN'), (',', '.')]

```

Joe met Joanne in Delhi .
→ [('Joe', 'NOUN'), ('met', 'VERB'), ('Joanne', 'NOUN'), ('in', 'ADP'), ('Delhi', 'NOUN'), ('.', '.')]]

Time flies like an arrow .
→ [('Time', 'NOUN'), ('flies', 'VERB'), ('like', 'ADP'), ('an', 'DET'), ('arrow', 'NOUN'), ('.', 'NOUN')]]

The good , the bad , the ugly went to a bar .
→ [('The', 'DET'), ('good', 'ADJ'), ('.', '.'), ('the', 'DET'), ('bad', 'ADJ'), ('.', '.'), ('the', 'DET'), ('ugly', 'ADJ'), ('went', 'VERB'), ('to', 'ADP'), ('a', 'DET'), ('bar', 'NOUN'), ('.', '.')]]

===== Unsupervised HMM Tagger =====

Today is a good day .
→ [('Today', 'ADP'), ('is', 'NOUN'), ('a', '.'), ('good', 'NUM'), ('day', 'ADV'), ('.', 'PRON')]]

Joe met Joanne in Delhi .
→ [('Joe', 'DET'), ('met', 'VERB'), ('Joanne', 'ADJ'), ('in', 'VERB'), ('Delhi', '.'), ('.', 'PRON')]]

Time flies like an arrow .
→ [('Time', 'ADJ'), ('flies', 'DET'), ('like', 'VERB'), ('an', '.'), ('arrow', 'NOUN'), ('.', 'NOUN')]]

The good , the bad , the ugly went to a bar .
→ [('The', 'ADP'), ('good', 'PRT'), ('.', 'PRON'), ('the', '.'), ('bad', 'NUM'), ('.', 'PRON'), ('the', 'NOUN'), ('ugly', '.'), ('went', 'PRT'), ('to', 'NUM'), ('a', 'ADV'), ('bar', 'ADV'), ('.', 'PRON')]]

===== Semi-supervised HMM Tagger =====

Today is a good day .
→ [('Today', 'PRON'), ('is', 'VERB'), ('a', 'DET'), ('good', 'ADJ'), ('day', 'NOUN'), ('.', '.')]]

Joe met Joanne in Delhi .
→ [('Joe', 'NOUN'), ('met', 'VERB'), ('Joanne', 'DET'), ('in', 'ADP'), ('Delhi', 'NOUN'), ('.', '.')]]

Time flies like an arrow .
→ [('Time', 'PRON'), ('flies', 'VERB'), ('like', 'ADP'), ('an', 'DET'), ('arrow', 'NOUN'), ('.', 'NOUN')]]

The good , the bad , the ugly went to a bar .
→ [('The', 'DET'), ('good', 'ADJ'), ('.', '.'), ('the', 'DET'), ('bad', 'ADV'), ('.', '.'), ('the', 'DET'), ('ugly', 'NUM'), ('went', 'VERB'), ('to',

```
'PRT'), ('a', 'DET'), ('bar', 'NUM'), ('.', '.')]


```

3 Task 2: Text Generation with HMMs (24 points)

In this part, you will use a trained HMM to generate text. The idea is to sample from the learned transition and emission probabilities to create new sequences.

3.1 2.1: Train HMM Model for Generation

First, train a supervised HMM model on the Brown corpus for text generation. We'll normalise words to lowercase for better generation.

```
[17]: def train_hmm_model(num_sentences=5000):
    """Train an HMM model on Brown corpus for text generation"""
    print("Loading Brown corpus...")
    tagged_sents = list(brown.tagged_sents(tagset="universal"))[:num_sentences]

    print(f"Training HMM on {len(tagged_sents)} sentences...")

    # Extract states and symbols
    all_tags = set()
    all_symbols = set()
    for sent in tagged_sents:
        for word, tag in sent:
            all_tags.add(tag)
            all_symbols.add(word.lower()) # Normalise to lowercase

    # Normalise training data to lowercase
    normalized_sents = []
    for sent in tagged_sents:
        normalized_sents.append([(word.lower(), tag) for word, tag in sent])

    # Train the model
    trainer = hmm.HiddenMarkovModelTrainer(
        states=list(all_tags),
        symbols=list(all_symbols)
    )

    tagger = trainer.train_supervised(normalized_sents)

    print("Training complete!")
    return tagger

# Train the model
tagger_gen = train_hmm_model(num_sentences=20000)
```

```
# Show model statistics
print(f"\nModel Statistics:")
print(f" - Number of states (POS tags): {len(tagger_gen._states)}")
print(f" - Number of symbols (words): {len(tagger_gen._symbols)}")
print(f" - States: {' '.join(sorted(tagger_gen._states))}")
```

Loading Brown corpus...

Training HMM on 20000 sentences...

Training complete!

Model Statistics:

- Number of states (POS tags): 12
- Number of symbols (words): 30743
- States: ., ADJ, ADP, ADV, CONJ, DET, NOUN, NUM, PRON, PRT, VERB, X

3.2 2.2: Implement State Sampling (7 points)

Implement the `sample_state()` function that samples the next POS tag based on transition probabilities.

Temperature parameter: Controls randomness - Lower temperature (e.g., 0.5): More conservative, follows high-probability transitions - Higher temperature (e.g., 2.0): More creative, explores diverse transitions

```
[18]: def sample_state(tagger, current_state, temperature=1.0):
    """
    Sample next state from transition distribution.

    Args:
        tagger: Trained HMM tagger
        current_state: Current POS tag
        temperature: Controls randomness (default 1.0)

    Returns:
        Next POS tag (state)
    """
    # TODO: Implement state sampling
    #
    # Steps:
    # 1. Get all possible states from tagger._states
    #
    # 2. For each state, get transition probability from current_state
    #    using: tagger._transitions[current_state].logprob(state)
    #
    # 3. Apply temperature scaling: probs = probs / temperature
    #
    # 4. Convert from log2 probabilities to regular probabilities: 2 ** probs
```

```

#
# 5. Normalise probabilities to sum to 1
#
# 6. Sample using np.random.choice(states, p=probs)

# 1. Get all possible states from tagger._states
all_states = list(tagger._states)
# 2. For each state, get transition probability from current_state
tprob=[]
for state in all_states:
    tprob.append(tagger._transitions[current_state].logprob(state))
# 3. Apply temperature scaling: probs = probs / temperature
tprob = np.array(tprob)
tprob = tprob / temperature
# 4. Convert from log2 probabilities to regular probabilities: 2 ** probs
tprob = 2 ** tprob

# 5. Normalise probabilities to sum to 1
if sum(tprob) != 0:
    tprob = tprob / sum(tprob)
else:
    tprob = np.ones(len(tprob)) / len(tprob)
# 6. Sample using np.random.choice(states, p=probs)
next_state = np.random.choice(all_states, p=tprob)
return next_state

```

3.3 2.3: Implement Word Sampling (7 points)

Implement the `sample_word()` function that samples a word given a POS tag based on emission probabilities.

```

[19]: def sample_word(tagger, state, temperature=1.0):
    """
    Sample word from output distribution of given state.

    Args:
        tagger: Trained HMM tagger
        state: Current POS tag
        temperature: Controls randomness (default 1.0)

    Returns:

```

```

Sampled word
"""
# TODO: Implement word sampling
#
# Steps:
# 1. Get all possible symbols (words) from tagger._symbols
#
# 2. For each symbol, get emission probability from state
#    using: tagger._outputs[state].logprob(symbol)
#
# 3. Apply temperature scaling: probs = probs / temperature
#
# 4. Convert from log2 probabilities to regular probabilities: 2 ** probs
#
# 5. Handle infinities (words with zero probability) using np.nan_to_num
#
# 6. Normalise probabilities to sum to 1
#    (if all are zero, use uniform distribution)
#
# 7. Sample using np.random.choice(symbols, p=probs)

# 1. Get all possible symbols (words) from tagger._symbols
symbols = list(tagger._symbols)
# 2) Get emission log2-probabilities for each symbol from this state
logprobs = []
for symbol in symbols:
    logprobs.append(tagger._outputs[state].logprob(symbol))
# 3. Apply temperature scaling: probs = probs / temperature
logprobs = np.array(logprobs)
logprobs = logprobs / temperature

# 4. Convert from log2 probabilities to regular probabilities: 2 ** probs
logprobs = 2 ** logprobs

# 5. Handle infinities (words with zero probability) using np.nan_to_num

logprobs = np.nan_to_num(logprobs, nan=0.0, posinf=0.0, neginf=0.0)

# 6. Normalise probabilities to sum to 1
if sum(logprobs) != 0:
    logprobs = logprobs / sum(logprobs)
else:
    logprobs = np.ones(len(logprobs)) / len(logprobs)

```

```
# 7. Sample using np.random.choice(symbols, p=probs)
word = np.random.choice(symbols, p=logprobs)
return word
```

3.4 2.4: Implement Text Generation (10 points)

Implement the `generate_text_from_word()` function that generates text starting from a given word.

```
[20]: def generate_text_from_word(tagger, start_word, length=20, temperature=1.0):
    """
    Generate text starting with a given word using the HMM model.

    Args:
        tagger: Trained HMM tagger
        start_word: Word to start the generation
        length: Number of words to generate
        temperature: Controls randomness (higher = more random)

    Returns:
        List of generated words
    """
    # TODO: Implement text generation
    #
    # Steps:
    # 1. Normalise start_word to lowercase
    #
    # 2. Check if start_word is in vocabulary (tagger._symbols)
    #    - If not, find similar words or use random word
    #
    # 3. Initialise generated list with start_word
    #
    # 4. Infer the most likely tag for start_word:
    #    - For each state, get emission probability
    #    - Choose state with highest probability
    #
    # 5. Generate subsequent words in a loop:
    #    - Sample next state using sample_state()
    #    - Sample word from that state using sample_word()
    #    - Append word to generated list
    #    - Update current_state
    #
    # 6. Return generated list
```

```

# 1. Normalise start_word to lowercase
# 1. Normalise start_word to lowercase
start_word = start_word.lower()
# 2. Check if start_word is in vocabulary (tagger._symbols)
symbols = list(tagger._symbols)
if start_word not in symbols:
    # Find similar words or use random word
    start_word = np.random.choice(symbols)
# 3. Initialise generated list with start_word
generated = [start_word]
# 4. Infer the most likely tag for start_word:
states = list(tagger._states)
best_state = None
best_prob = -10000000000000000
for state in states:
    logprob = tagger._outputs[state].logprob(start_word)
    if logprob > best_prob:
        best_prob = logprob
        best_state = state

# 5. Generate subsequent words in a loop:
for i in range(length - 1):
    next_state = sample_state(tagger, best_state, temperature=temperature)
    next_word = sample_word(tagger, next_state, temperature=temperature)

    generated.append(next_word)
    best_state = next_state

# 6) Return generated list
return generated

```

3.5 2.6: Test Text Generation

Now test your text generation functions with different starting words and temperatures.

```

[21]: # Test with example words
print("="*70)
print("EXAMPLE GENERATIONS")
print("="*70)

example_words = ["the", "dog", "running", "beautiful", "computer", "yesterday"]

for word in example_words:

```

```

print(f"\n--- Starting with '{word}' ---")
try:
    words = generate_text_from_word(tagger_gen, word, length=15,
    ↪temperature=1.0)
    print(" ".join(words))
except Exception as e:
    print(f"Error: {e}")

```

EXAMPLE GENERATIONS

```

--- Starting with 'the' ---
the project , inmates room , brahms in tone consumed the surprise have a pen

--- Starting with 'dog' ---
dog `` and anton than will to became friday derive the pay with 5 ,

--- Starting with 'running' ---
running club st. to more society bathroom greenwich in president , . the
interior susan

--- Starting with 'beautiful' ---
beautiful st. , of fancy of much 72nd illustration terms has in a nina are

--- Starting with 'computer' ---
computer than a largest ' from honoured miss on discretion and family by any
roads

--- Starting with 'yesterday' ---
yesterday in the question ' resident in richard that breaking fail that posted
scabbard are

```

```

[22]: # Test different temperatures
print("\n" + "="*70)
print("TEMPERATURE COMPARISON")
print("="*70)

test_word = "the"
temperatures = [("Conservative", 0.5), ("Balanced", 1.0), ("Creative", 2.0)]

for temp_name, temp_value in temperatures:
    print(f"\n{temp_name} (temp={temp_value}):")
    words = generate_text_from_word(tagger_gen, test_word, length=20,
    ↪temperature=temp_value)
    print(" ".join(words))

```

=====

TEMPERATURE COMPARISON

=====

Conservative (temp=0.5):

the mr. . only few city in the good christ of the side of the side of the side
``

Balanced (temp=1.0):

the corn kaleidoscope , and whole model , and beginning were he , '' and
reasoning continue to as one

Creative (temp=2.0):

the galtier's west 1769-1842 designed over alvin 1.24 or extraordinary vests on
there start above 4-13 he'd ' , 12

Question 2.1: How does the temperature parameter affect the quality and creativity of generated text? Provide specific examples from your outputs. (4 points, 3-5 sentences)

[In temperature 0.5, model is conservative and repetitive text such as of the side of the side, so reduces creativity. In temperature 1, model is balance. It has more varied vocabulary and a reasonable sentence structure in phrases like “the corn kaleidoscope , and whole model”. In temture 2. model creats more creative but less coherent text such as “1769–1842”, “1.24”, “4-13”]

4 Convert Your Colab Notebook to PDF

4.0.1 Step 1: Download Your Notebook

- Go to **File** → **Download** → **Download .ipynb**
- Save the file to your computer

4.0.2 Step 2: Upload to Colab

- Click the **folder icon** on the left sidebar
- Click the **upload button**
- Select your downloaded .ipynb file
- Wait for the upload to complete

4.0.3 Step 3: Run the Code Below

- **Uncomment the cell below** and run the cell
- This will take about 1-2 minutes to install required packages

4.0.4 Step 4: Enter Notebook Name

- When prompted, type your notebook name (e.g. `gs_000000_as2.ipynb`)
- Press Enter

4.0.5 The PDF will automatically download to your computer

```
[ ]: # # Install required packages (this takes about 30 seconds)
print("Installing PDF converter... please wait...")
!apt-get update -qq
!apt-get install -y texlive-xetex texlive-fonts-recommended_
↳texlive-plain-generic pandoc > /dev/null 2>&1
!pip install -q nbconvert

print("\n" + "="*50)
print("COLAB NOTEBOOK TO PDF CONVERTER")
print("="*50)
print("\nSTEP 1: Download your notebook")
print("- Go to File → Download → Download .ipynb")
print("- Save it to your computer")
print("\nSTEP 2: Upload it here")
print("- Click the folder icon on the left ( )")
print("- Click the upload button and select your .ipynb file")
print("- Wait for upload to complete")
print("\nSTEP 3: Enter the filename below")
print("="*50)

# # Get notebook name from user
notebook_name = input("\nEnter your notebook name: ")

# # Add .ipynb if missing
if not notebook_name.endswith('.ipynb'):
    notebook_name += '.ipynb'

import os
notebook_path = f'/content/{notebook_name}'

# # Check if file exists
if not os.path.exists(notebook_path):
    print(f"\n Error: '{notebook_name}' not found in /content/")
    print("\nMake sure you uploaded the file using the folder icon ( ) on the_
↳left!")
else:
    print(f"\n Found {notebook_name}")
    print("Converting to PDF... this may take 1-2 minutes...\n")

# # Convert the notebook to PDF
!jupyter nbconvert --to pdf "{notebook_path}"

# # Download the PDF
from google.colab import files
pdf_name = notebook_name.replace('.ipynb', '.pdf')
```

```
pdf_path = f'/content/{pdf_name}'

if os.path.exists(pdf_path):
    print(" SUCCESS! Downloading your PDF now...")
    files.download(pdf_path)
    print("\n Done! Check your downloads folder.")
else:
    print(" Error: Could not create PDF")
```

Installing PDF converter... please wait...

W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)